Gazi University Department of Computer Engineering

CENG-479 Parallel Programming

"CUDA-Accelerated SIFT Feature Detection and FLANN-Based Descriptor Matching on ROS Image Streams"

Project Proposal

Ceren Akmeşe - 22118080701

Said Berk - 2111808070

Spring 2026

1. Project Title and Team Members

1.1. Project Title:

CUDA-Accelerated SIFT Feature Detection and FLANN-Based Descriptor Matching on ROS Image Streams

1.2. Team Members:

Ceren Akmeşe
Said Berk

1.3. Course:

CENG-479 Parallel Programming

1.4. Selected Technology:

CUDA
ROS

2. Problem Statement

Finding matching points between two photographs of the same scene is one of the most fundamental operations in computer vision. Whether the application is stitching holiday photos into a panorama, reconstructing a 3D model from drone footage, or localising a robot inside a building, the pipeline almost always starts the same way: detect distinctive keypoints in each image, describe them with a compact numerical fingerprint, and then match the fingerprints across the two views. The Scale-Invariant Feature Transform — SIFT — has been the workhorse algorithm for this task since Lowe (2004) published his landmark paper, and it remains a standard benchmark against which newer learned approaches are evaluated.

The problem is that SIFT is slow. Building the Gaussian scale-space pyramid, detecting extrema across octaves, assigning orientations, and computing 128-dimensional gradient histogram descriptors involves a large number of floating-point operations per image. On a modern laptop CPU running a single thread, a full HD image pair can take several hundred milliseconds to process end-to-end, which rules out real-time use without some form of acceleration.

In robotics applications, images are rarely processed as static file pairs. Instead, they arrive as a continuous stream of sensor messages over ROS (Robot Operating System) topics — specifically as `sensor_msgs/Image` messages published by camera drivers. This introduces a distinct I/O constraint: images must be received, converted, and dispatched to the GPU quickly enough not to create a processing backlog. For this reason, the CPU-side I/O path — including ROS message deserialization and host-to-device memory transfer — is treated as a first-class concern in this project's design, not an afterthought.

Once descriptors have been extracted, matching them introduces a second bottleneck. Comparing every descriptor from image A against every descriptor from image B is quadratic in the number of

keypoints. FLANN, the Fast Library for Approximate Nearest Neighbors introduced by Muja and Lowe (2009), addresses this with randomized k-d trees that answer nearest-neighbor queries in sub-linear expected time at a small and controllable loss of accuracy. The distance computations inside the search structure reduce to repeated vector dot products, which are exactly the kind of arithmetic that GPU cores are designed to perform in bulk.

This project will implement a CUDA GPU version of the full SIFT-FLANN pipeline, validated against OpenCV's reference CPU implementation. The goal is to measure the speedup that GPU parallelism delivers at each stage and to understand where the bottlenecks lie — including the communication overhead introduced by the ROS messaging layer.

3. Motivation: Why This Problem Benefits from Parallelism

The SIFT pipeline contains several stages with fundamentally different parallel structures, and that variety is part of what makes it an interesting vehicle for a parallel programming course project.

The Gaussian convolution that builds the scale-space pyramid is memory-bandwidth-bound and embarrassingly parallel at the pixel level. Every output sample depends only on a spatially local neighbourhood of the input, so millions of pixels can be convolved simultaneously. The formula applied at each location is a weighted sum:

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

where G is a Gaussian kernel of standard deviation σ and I is the input image. Because each output pixel depends only on its own neighbourhood, all pixels can be computed in parallel with no global synchronization. The same data-parallel structure applies to the Difference-of-Gaussians subtraction step that follows.

Extremum detection requires each candidate sample to be compared against 26 neighbours across two adjacent scale levels, but the comparison for one sample is completely independent of every other sample. Descriptor computation is more expensive per keypoint but similarly parallel: the 128 histogram bins of each descriptor can be accumulated independently across threads within a block, and different keypoints can be processed by different blocks simultaneously.

The matching stage reduces to computing L2 distances between 128-dimensional floating-point vectors. GPUs achieve near-peak throughput on large matrix products, so this stage is also well-suited to GPU acceleration. A summary of why parallelism works well at each stage:

- Data parallelism in convolution: every pixel computes its Gaussian-blurred value independently from all others.
- Independent extremum tests: each candidate location is assessed without knowledge of any other location's result.
- Block-level cooperation in descriptors: threads within one CUDA block collaborate via shared memory to accumulate one keypoint's histogram, and blocks for different keypoints run fully in parallel.
- Matrix-multiplication structure in matching: L2 distance computation maps onto batched GEMM operations, giving access to highly optimised cuBLAS routines.
- ROS I/O bottleneck awareness: the CPU-side message reception and host-to-device transfer are measured explicitly, since in a streaming robotics context these costs directly affect real-world throughput.

4. Proposed Parallel Algorithm

4.1 Input and Output

Images arrive as sensor_msgs/Image ROS messages subscribed from a camera topic. On the CPU side, each incoming message is converted from the ROS message buffer to a plain grayscale float32 array using cv_bridge, then transferred to device memory via a pinned-memory staging buffer to minimise latency. All further SIFT processing happens entirely on the GPU. The output — a list of matched keypoint pairs together with per-stage timing measurements — is copied back to the host and published on a ROS result topic for downstream consumers. We will test with three image resolutions: 640×480, 1280×720, and 1920×1080.

4.2 CPU Reference Baseline

Rather than implementing a full sequential SIFT pipeline from scratch — which would consume most of the available development time without adding insight about GPU programming — we use **OpenCV's built-in cv::SIFT** implementation as the golden reference. This is the same implementation used throughout the computer vision community and gives us a numerically reliable, well-tested ground truth against which to validate CUDA outputs. The CPU baseline reads both images via cv_bridge, runs cv::SIFT::detectAndCompute, performs FLANN-based matching using OpenCV's cv::FlannBasedMatcher, and applies Lowe's ratio test. It serves two purposes: correctness verification for the GPU outputs, and a fair denominator for all speedup calculations.

4.3 CUDA Parallel Version

The CUDA version organises the pipeline as a sequence of kernel launches, keeping all intermediate data on the GPU between stages. The stages are:

- Gaussian Convolution: a separable approach with a horizontal pass followed by a vertical pass, each with one thread per output pixel. The vertical pass uses shared memory tiling with a halo region to avoid redundant global memory reads.
- Difference-of-Gaussians: a simple element-wise subtraction kernel applied across adjacent scale levels.
- Extremum Detection: one thread per candidate pixel, comparing each against 26 neighbours across adjacent scale levels.
- Keypoint Compaction: surviving keypoints are compacted using NVIDIA Thrust's thrust::copy_if, which implements a prefix-sum compaction efficiently and avoids the synchronization complexity of a hand-written custom kernel.
- Descriptor Computation: one thread block per keypoint, with threads cooperating through global memory to accumulate the 128-bin gradient histogram. Each block processes one keypoint independently.
- Matching: L2 distance computation is expressed as a batched matrix multiplication dispatched through cuBLAS SGEMM. Lowe's ratio test runs on the device, so only accepted match pairs are copied back to the host.

A representative sketch of the convolution kernel:

```
CUDA Kernel GaussConv_Horizontal(Input, Output, kernel, width, height, kRadius):
   x = blockIdx.x * blockDim.x + threadIdx.x
   y = blockIdx.y * blockDim.y + threadIdx.y
   if x >= width or y >= height: return
   sum = 0.0
   for k = -kRadius to kRadius:
      cx = clamp(x + k, 0, width - 1)
```

```
    sum += Input[y * width + cx] * kernel[k + kRadius]
Output[y * width + x] = sum
```

4.4 ROS Integration and I/O Design

Because images arrive from a live ROS topic rather than from disk, CPU-side I/O is a latency-sensitive path. The integration design prioritises minimal copying between the ROS message buffer and device memory:

- A ROS subscriber listens on the camera image topic. The cv_bridge::toCvShare call avoids an extra copy by wrapping the message buffer without allocating new memory.
- The resulting cv::Mat is converted to float32 and placed into a pinned (page-locked) host buffer, from which cudaMemcpyAsync transfers it to device memory asynchronously to overlap communication with computation.
- Result keypoint pairs are published on a separate ROS topic so downstream nodes (e.g. a localisation or mapping module) can consume them without polling.
- Per-message latency and throughput are logged alongside per-stage GPU timing so that the contribution of ROS I/O to total pipeline time is clearly visible in the results.

4.5 Correctness Strategy

We will compare CUDA outputs directly against the OpenCV CPU reference. For descriptor vectors we will report maximum and average absolute difference per element. For the final match list we will report the fraction of GPU matches that agree with CPU matches at the same keypoint locations, using a position tolerance of 1.5 pixels and a descriptor distance tolerance of 1×10^{-3}. Small floating-point differences arising from GPU FMA operations are expected and acceptable; what matters is that the two pipelines produce the same set of matches for the same input.

5. CUDA Implementation

CUDA is the natural choice for this project because each stage of the SIFT-FLANN pipeline exposes a different flavour of data parallelism that maps cleanly onto GPU hardware. Gaussian convolution is memory-bandwidth-bound; the GPU's high-bandwidth GDDR6 memory and coalescing-friendly thread layout make it far more efficient than a single CPU core for this step. Descriptor computation is arithmetic-intensive; thousands of CUDA cores can accumulate histogram bins simultaneously. The matching stage is dominated by large matrix operations, which are exactly what cuBLAS is optimised for.

All CUDA code will be written in CUDA C/C++ and compiled with nvcc. We will use CUDA events for precise kernel-level timing and wall-clock measurements for end-to-end pipeline time including ROS message reception and host-to-device transfers.

- CUDA kernels will handle convolution, DoG computation, extremum detection, descriptor computation, and distance-based matching.
- OpenCV cv::SIFT serves as the CPU reference baseline; no sequential SIFT is implemented from scratch.
- NVIDIA Thrust is used for keypoint compaction (thrust::copy_if) to avoid low-level prefix-sum implementation risk.
- cuBLAS SGEMM handles the batched L2 distance computations in the matching stage.
- cv_bridge is used for ROS-to-OpenCV conversion on the CPU side, with pinned memory for fast host-to-device transfers.

- CUDA events separately measure each kernel's execution time and the total memory transfer cost, including the ROS I/O path.
- We will experiment with block sizes of 16×16 and 32×8 for the convolution kernels to understand how thread layout affects occupancy and memory coalescing.

6. Performance Analysis Plan

The central metric is speedup, defined as:

Speedup = OpenCV CPU Time / CUDA GPU Time (kernel-only)

We will report two flavours of CUDA time: kernel-only time and end-to-end time. Kernel-only time isolates the computational gain from GPU parallelism. End-to-end time is more realistic for real-world robotics use because it includes ROS message reception, host-to-device and device-to-host memory transfers, and result publishing. The gap between these two numbers is expected to be informative, particularly at lower resolutions where transfer overhead dominates.

Metric	Planned Measurement
Input size	640×480, 1280×720, and 1920×1080 image pairs from ROS topics
Versions compared	OpenCV CPU reference, CUDA baseline kernel version
Timing metrics	ROS I/O time, H2D transfer, per-stage kernel time, D2H transfer, total end-to-end time
Throughput	Keypoints processed per second, matches produced per second
Correctness metrics	Descriptor element difference vs. OpenCV reference, match list agreement percentage
Scalability factors	Image resolution, CUDA block size

6.1. Project Scope and Feasibility

The scope is intentionally constrained to what can be implemented, validated, and measured within the available time. The CPU baseline is provided by OpenCV rather than written from scratch. Keypoint compaction uses Thrust rather than a custom prefix-sum kernel. The descriptor kernel targets correctness first using global memory; shared-memory optimisation is deferred. ROS integration is kept minimal: one subscriber, one publisher, pinned-memory transfer — no multi-node orchestration.

Scope Category	Description
Required	OpenCV CPU reference baseline, ROS subscriber/publisher with cv_bridge integration, CUDA convolution and DoG kernels, extremum detection, Thrust-based compaction, descriptor computation (global memory), cuBLAS matching, correctness comparison, timing results including ROS I/O
Extension (time permitting)	Shared-memory optimised descriptor kernel; visualisation of matched keypoints overlaid on image pair and published as a ROS Image topic
Out of scope	Full SLAM pipeline, learned feature descriptors, real-time video processing at high frame rates, multi-camera setups

6.2. Risks and Mitigation

- Risk: Variable keypoint count makes GPU output buffer sizing non-trivial. Mitigation: Thrust thrust::copy_if compacts surviving keypoints into a known-size array without manual prefix-sum logic or over-allocation.
- Risk: Floating-point differences between OpenCV CPU and GPU results due to FMA and rounding order. Mitigation: Tolerance-based numerical comparison; descriptor mismatch reported as mean absolute error rather than requiring bit-exact agreement.
- Risk: Memory transfer overhead — including the ROS I/O path — may reduce end-to-end GPU speedup, especially at 640×480. Mitigation: Report both kernel-only and end-to-end timing and discuss the difference openly, including the contribution of ROS deserialization.
- Risk: Warp divergence in the descriptor kernel due to varying keypoint orientations. Mitigation: Sort keypoints by image position before the descriptor kernel launch to group geometrically similar patches in the same warp.
- Risk: ROS callback timing jitter may distort per-message latency measurements. Mitigation: Log timestamps at message arrival and at each pipeline stage boundary; report mean and standard deviation across a batch of frames rather than a single measurement.
- Risk: Scope creep. Mitigation: Deliverable stays strictly limited to detection, description, and matching. No RANSAC or downstream geometry.

6.3. Expected Contribution

By the end of this project, we will have a working and measurable CUDA implementation of the SIFT-FLANN pipeline, benchmarked honestly against the OpenCV CPU reference at multiple image resolutions. The implementation handles real ROS image messages from end to end, making the timing results relevant to actual robotics use cases. More broadly, the project will demonstrate how a multi-stage computer vision algorithm can be decomposed into parallel work units of different types — memory-bandwidth-bound convolutions, compute-intensive histogram accumulations, and arithmetic-heavy distance computations — and how each type can be mapped onto GPU hardware efficiently.

The final report will cover algorithm design and implementation, correctness validation against OpenCV, per-stage and end-to-end timing results including ROS I/O costs, speedup analysis, and an honest discussion of memory-transfer effects and the limits of GPU acceleration for this class of problem.

7. Academic References

Fabijańska, A. (2018). Speeding up SIFT implementation on NVIDIA CUDA. EURASIP Journal on Image and Video Processing, 2018(1), 1–14. https://doi.org/10.1186/s13640-018-0305-6

Lowe, D. G. (1999). Object recognition from local scale-invariant features. In Proceedings of the 7th IEEE International Conference on Computer Vision (Vol. 2, pp. 1150–1157). IEEE. https://doi.org/10.1109/ICCV.1999.790410

Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. International Journal of Computer Vision, 60(2), 91–110. https://doi.org/10.1023/B:VISI.0000029664.99615.94

Muja, M., & Lowe, D. G. (2009). Fast approximate nearest neighbors with automatic algorithm configuration. In A. Ranchordas & H. Araújo (Eds.), Proceedings of the International Conference on Computer Vision Theory and Applications (Vol. 1, pp. 331–340). INSTICC. https://doi.org/10.5220/0001787803310340

Muja, M., & Lowe, D. G. (2014). Scalable nearest neighbor algorithms for high dimensional data. IEEE Transactions on Pattern Analysis and Machine Intelligence, 36(11), 2227–2240. https://doi.org/10.1109/TPAMI.2014.2321376

NVIDIA Corporation. (2023). CUDA C++ programming guide (Version 12.3). https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., & Ng, A. Y. (2009). ROS: An open-source Robot Operating System. ICRA Workshop on Open Source Software.

Wu, C. (2007). SiftGPU: A GPU implementation of scale invariant feature transform (SIFT). University of North Carolina at Chapel Hill. http://cs.unc.edu/~ccwu/siftgpu/